

Knowledge Base

Synthetic Financial Time Series Generation

From Arithmetic to GANs — Every Step Explained

Feynman Technique Throughout

Situation → Problem → Worked Arithmetic
→ Formal Name → Where the Analogy Breaks → Why It Matters Here

BSE Master Thesis 2026 · Victor Sobottka

Contents

1	Arithmetic Foundations	1
1.1	The Logarithm	1
1.2	Mean and Standard Deviation	2
1.3	The Normal Distribution and Why It Fails for Markets	3
1.4	Conditional Probability	4
2	Time Series Analysis	6
2.1	Stationarity	6
2.2	Autocorrelation Function (ACF)	7
2.3	GARCH: The Volatility Clustering Model	8
2.4	ARCH Test	9
2.5	Hurst Exponent and Long Memory	10
3	Heavy Tails and Power Laws	12
3.1	Power Laws	12
3.2	Why Kurtosis Is Unreliable for BRICS Markets	13
3.3	Wasserstein Distance	14
4	Generative Adversarial Networks	16
4.1	The GAN Idea	16
4.2	WGAN and the Wasserstein Distance	17
4.3	TimeGAN	18
4.4	QuantGAN	19
4.5	FinGAN	19
5	Evaluation: Metrics, Taxonomy, and the Shuffled-Control Audit	21
5.1	The Metric Taxonomy	21
5.2	The Shuffled-Control Audit	22
5.3	Discriminative AUC: Null Calibration and Direction Bug	23
6	Downstream Utility: VaR Backtesting and QLIKE	25
6.1	Value at Risk	25

6.2	Kupiec Backtest	26
6.3	Christoffersen Independence Test	27
6.4	QLIKE Loss	27
6.5	Walk-Forward Validation	28
	Symbol Glossary	30

Chapter 1

Arithmetic Foundations

Everything in this thesis rests on three operations: logarithms, percentages, and the normal distribution. This chapter builds each one from first principles using the Feynman technique.

1.1 The Logarithm

Situation — the everyday picture

You invest \$100. After one year your account shows \$108. After two years, \$116.64. If someone asks “how fast did your money grow?”, you could say “8%/year”, or you could say “0.0770/year in log units”. Both answers are correct. The second is more useful for calculations.

Problem — where it breaks or why it matters

Percentages do not add up correctly across time periods. A 10% gain followed by a 10% loss gives $1.10 \times 0.90 = 0.99$ — a 1% net loss, not 0. Logarithms turn multiplication into addition, making multi-period returns additive.

Worked Arithmetic — concrete numbers

Day 1 price: 4,000. Day 2 price: 4,080. Day 3 price: 4,050.

Log return, day 1→2:

$$r_1 = \ln\left(\frac{4080}{4000}\right) = \ln(1.02) \approx 0.0198 \approx 1.98\%$$

Log return, day 2→3:

$$r_2 = \ln\left(\frac{4050}{4080}\right) \approx -0.0074 \approx -0.74\%$$

Two-day total by addition: $r_1 + r_2 = 0.0198 - 0.0074 = 0.0124$

Direct calculation: $\ln(4050/4000) \approx 0.0124$ ✓

The sum of daily log returns equals the log return over the whole period. For percentage returns: $(+2\%) + (-0.74\%) = +1.26\%$, but the real answer is $+1.24\%$. The error is small for small moves, large for large moves.

Formal Name — the mathematics

The natural logarithm $\ln(x)$ is the unique function satisfying:

$$e^{\ln(x)} = x, \quad \ln(e^x) = x, \quad \ln(xy) = \ln(x) + \ln(y)$$

where $e \approx 2.71828$. The log return is defined as:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

The key property: $\sum_{t=1}^T r_t = \ln(P_T/P_0)$ exactly.

Where the Analogy Breaks

Logs work perfectly for prices, but they require strictly positive inputs. A price can never be exactly zero (a market would cease trading), so this is rarely binding. However, log returns of -100% correspond to $P_t = 0$, which $\ln$ maps to $-\infty$. In practice, we never observe this.

Why It Matters Here — link to the thesis

Every number in this thesis — every metric, every model input, every evaluation — is computed from daily log returns r_t . We never use raw prices because they are non-stationary (trending over time) and incomparable across markets.

1.2 Mean and Standard Deviation

Situation — the everyday picture

You measure the temperature every day for a week: 15, 17, 14, 19, 16, 21, 13 degrees. You want two summaries: the “typical” value, and how much the values vary.

Problem — where it breaks or why it matters

One number never tells the whole story. Knowing the average temperature of 16.4°C does not tell you that some days hit 21° . We need both location and spread.

Worked Arithmetic — concrete numbers

Seven values: $x = \{15, 17, 14, 19, 16, 21, 13\}$.

Mean: $\bar{x} = (15 + 17 + 14 + 19 + 16 + 21 + 13)/7 = 115/7 \approx 16.43$

Deviations: $\{-1.43, +0.57, -2.43, +2.57, -0.43, +4.57, -3.43\}$

Squared deviations: $\{2.04, 0.33, 5.90, 6.60, 0.18, 20.87, 11.77\}$

Variance: $s^2 = (2.04 + 0.33 + \dots)/6 = 47.69/6 = 7.95$ (divide by $n - 1$ for sample)

Standard deviation: $s = \sqrt{7.95} \approx 2.82^\circ\text{C}$

Formal Name — the mathematics

For a sample $\{x_1, \dots, x_n\}$:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad s = \sqrt{s^2}$$

The $(n - 1)$ denominator corrects for the fact that we estimated $\bar{x}$ from the same sample (Bessel's correction). The population versions use n .

Where the Analogy Breaks

Standard deviation describes spread around the mean, but only makes full sense when the distribution is symmetric. For skewed distributions (like stock returns, which can fall much further than they rise), standard deviation understates downside risk.

Why It Matters Here — [link to the thesis](#)

Daily volatility $\sigma = \text{sd}(r_t)$ is the foundational measure in all five markets. In our 20-year data: MOEX $\sigma = 1.87\%$ /day, BOVESPA 1.63%. Annualised: multiply by $\sqrt{252}$. Standard deviation also enters GARCH, VaR, discriminative AUC, and every distance metric we compute.

1.3 The Normal Distribution and Why It Fails for Markets

Situation — the everyday picture

Roll a fair die 100 times and record the average result. Do this experiment 10,000 times. Plot the 10,000 averages. They form a perfect bell shape: most near 3.5, rarely below 2.5 or above 4.5. This is the Central Limit Theorem in action.

Problem — where it breaks or why it matters

The normal distribution makes precise predictions about extreme events. For a bell curve, a five-standard-deviation event should occur once every 6,922 years. We observed 76 such events across 24,758 BRICS market-days. The model is not slightly wrong; it is orders-of-magnitude wrong.

Worked Arithmetic — concrete numbers

The Gaussian probability of $|x| > 5\sigma$ is:

$$\mathbb{P}(|Z| > 5) = 2 \times \Phi(-5) \approx 5.73 \times 10^{-7}$$

With 252 trading days per year, the expected waiting time is:

$$\text{Wait} = \frac{1}{252 \times 5.73 \times 10^{-7}} \approx 6,922 \text{ years}$$

We have 24,758 market-days across 5 markets ≈ 100 market-years total. The Gaussian predicts $100 \times 252/6,922 \approx 0.014$ such days. We observed 76. The ratio is $76/0.014 \approx 5,400$.

Formal Name — the mathematics

The normal distribution $\mathcal{N}(\mu, \sigma^2)$ has PDF:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

The **excess kurtosis** measures tail weight relative to the normal:

$$\text{Kurt} = \frac{\mathbb{E}[(X - \mu)^4]}{(\mathbb{E}[(X - \mu)^2])^2} - 3$$

Normal: Kurt = 0. Heavy-tailed distributions: Kurt > 0. BRICS markets: 5.73–65.70.

Important caveat: kurtosis requires $\mathbb{E}[X^4] < \infty$. For power-law tails $P(|X| > x) \sim x^{-\alpha}$, the k -th moment exists only if $k < \alpha$. With BRICS tail indices $\hat{\alpha} = 2.54\text{--}3.21$, the fourth moment is infinite everywhere and the third moment is infinite in 4 of 5 markets. Sample kurtosis diverges with n rather than converging.

Where the Analogy Breaks

The Central Limit Theorem says averages of independent identically distributed random variables converge to a normal. Market returns are *not* independent (volatility clusters), and their distribution is *not* finite-variance when $\alpha < 2$ (our markets have $\alpha > 2$, so variance is finite, but fourth moments are not). CLT still applies to averages, but individual extreme events are not well-described by the normal.

Why It Matters Here — [link to the thesis](#)

The failure of the normal distribution is the entire motivation for this thesis. If market returns were Gaussian, banks would be adequately protected by Gaussian VaR models. Because they are not, we need generative models that reproduce heavy tails and volatility clustering — and a rigorous evaluation framework to verify they do.

1.4 Conditional Probability

Situation — the everyday picture

A city has 60% chance of rain on any day. But if it rained yesterday, the chance of rain today is 80%. Knowing yesterday's outcome changes the probability of today's.

Worked Arithmetic — concrete numbers

Let A = “big market move today”, B = “big market move yesterday”.

MOEX 20-year data: $\mathbb{P}(A) = 3.8\%$, $\mathbb{P}(A|B) = 22.9\%$.

Ratio: $\mathbb{P}(A|B)/\mathbb{P}(A) = 22.9/3.8 = 6.1$.

This means: knowing that yesterday was a big-move day makes today's big-move probability **6.1×** higher than the unconditional baseline.

Formal Name — the mathematics

$$\mathbb{P}(A | B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$$

Events A and B are **independent** if $\mathbb{P}(A|B) = \mathbb{P}(A)$. Market returns are not independent: $\mathbb{P}(A|B) \neq \mathbb{P}(A)$. The ratio $\mathbb{P}(A|B)/\mathbb{P}(A) = 6.1$ measures the strength of **volatility clustering**.

Where the Analogy Breaks

Conditional probability does not imply causality. MOEX big moves after invasion news are driven by the news, not by the prior day's move per se. Correlation in extremes is not the same as predictability of direction.

Why It Matters Here — [link to the thesis](#)

The shuffled-control experiment destroys conditional structure: $\mathbb{P}(A|B)_{\text{shuffled}} \approx \mathbb{P}(A)_{\text{shuffled}}$ after permutation. Any generator that reproduces only the marginal distribution $\mathbb{P}(A)$ but not the conditional $\mathbb{P}(A|B)$ will pass distributional tests but fail temporal ones.

Chapter 2

Time Series Analysis

A **time series** is a sequence of measurements indexed by time: $r_1, r_2, \dots, r_T$. Market returns form a time series. This chapter explains the tools we use to characterise their structure.

2.1 Stationarity

Situation — the everyday picture

The temperature in Barcelona fluctuates around 18°C year-round (ignoring seasons). The sea level has been rising 3mm/year for decades. The temperature process is *stationary* — its statistical properties do not change over time. Sea level is *non-stationary* — its mean is drifting.

Problem — where it breaks or why it matters

If a time series is non-stationary, any statistics we compute (mean, variance, correlations) depend on when we measured them, making them meaningless for predicting the future. All our models assume stationarity of log returns.

Formal Name — the mathematics

A process $\{X_t\}$ is **weakly stationary** if:

1. $\mathbb{E}[X_t] = \mu$ (constant mean)
2. $\text{Var}(X_t) = \sigma^2 < \infty$ (constant finite variance)
3. $\text{Cov}(X_t, X_{t+k}) = \gamma(k)$ (covariance depends only on lag k , not t)

Raw price levels P_t are non-stationary (trending). Log returns $r_t = \ln(P_t/P_{t-1})$ are approximately stationary.

Where the Analogy Breaks

“Stationarity” in the weak sense does not require the distribution to be normal, only that the first two moments are constant. BRICS markets have constant mean (near zero) and stable variance in the long run, but their higher moments (skewness, kurtosis) exhibit regime changes (e.g., the MOEX invasion event).

Why It Matters Here — link to the thesis

All evaluation metrics in this thesis assume stationarity: the Wasserstein distance, ACF-MAE, Hurst exponent, and discriminative AUC are all computed on the assumption that the distribution of r_t does not change between train and test. The temporal 80/10/10 split (no shuffling) is the correct design for non-stationary data.

2.2 Autocorrelation Function (ACF)**Situation — the everyday picture**

You write down today's temperature and tomorrow's temperature for 365 days. You plot them as (x_t, x_{t+1}) pairs. Because temperature is persistent (a hot day is likely to be followed by another hot day), the scatter plot shows a positive slope. The *correlation* between today's and tomorrow's temperature is around +0.8.

Now do the same for today's and next week's temperature: the correlation is lower, maybe +0.4. After a month, it might be nearly zero — last month's weather tells you little about today.

Problem — where it breaks or why it matters

For market returns, the correlation between r_t and r_{t+1} is *near zero* for all lags (fact 2: markets are approximately unpredictable). But the correlation between $|r_t|$ and $|r_{t+1}|$ — the absolute return, a measure of how wild each day was — is *significantly positive* for lags of 1 to 100 or more (volatility clustering).

Worked Arithmetic — concrete numbers

BOVESPA 20-year data ($n = 4,953$):

Lag k	$\hat{\rho}(r, k)$	$\hat{\rho}(r , k)$	$\hat{\rho}(r^2, k)$
1	+0.03	+0.19	+0.17
5	−0.01	+0.14	+0.12
20	+0.01	+0.09	+0.08
50	≈ 0	+0.05	+0.04

The raw return autocorrelation is near zero (unpredictable direction). The absolute and squared return autocorrelation is positive and decays slowly (persistent volatility). A good generator must produce synthetic $|r_t|$ sequences with the *same* slow decay.

Formal Name — the mathematics

The **sample autocorrelation** at lag k is:

$$\hat{\rho}(k) = \frac{\sum_{t=1}^{T-k} (x_t - \bar{x})(x_{t+k} - \bar{x})}{\sum_{t=1}^T (x_t - \bar{x})^2}$$

The **Autocorrelation Function** (ACF) is the sequence $\{\hat{\rho}(0), \hat{\rho}(1), \hat{\rho}(2), \dots\}$. Our metric **ACF-MAE** compares real and synthetic ACF vectors:

$$\text{ACF-MAE}(f) = \frac{1}{K} \sum_{k=1}^K |\hat{\rho}_{\text{real}}(k) - \hat{\rho}_{\text{syn}}(k)|, \quad K = 20$$

We compute this for three transforms: raw returns r_t , absolute returns $|r_t|$, squared returns r_t^2 .

Where the Analogy Breaks

ACF-MAE measures average discrepancy, which can miss systematic biases at specific lags. However, it has the crucial property of being *continuous with no saturation*: unlike ARCH-LM, it does not underflow to 0 when applied to real fat-tailed data.

Why It Matters Here — [link to the thesis](#)

ACF-MAE on $|r_t|$ and r_t^2 is the *primary* ranking signal for volatility clustering (Temporal family). The shuffled control scores 0.043 on ACF(returns) and 0.076 on ACF($|r|$) — correctly penalised. Any generator that correctly reproduces clustering will have smaller ACF-MAE than the shuffled control on these metrics.

2.3 GARCH: The Volatility Clustering Model

Situation — the everyday picture

Think of the stock market as a mood thermometer. On quiet days, it reads “calm” and barely moves. On anxious days, it reads “stormy” and moves a lot. The key insight: the mood today is heavily influenced by the mood yesterday. If yesterday was stormy, today is probably also stormy.

Problem — where it breaks or why it matters

The normal distribution assigns the same probability to a 2% move on every day. Real markets have quiet periods where 0.5% moves are normal and turbulent periods where 2% moves are common. A model that ignores this will badly misjudge risk on quiet days (underestimating tail risk) and turbulent days (overestimating) simultaneously.

Worked Arithmetic — concrete numbers

GARCH(1,1) fit to BOVESPA (20-year):

$\omega = 0.000002$, $\alpha = 0.08$, $\beta = 0.91$

Long-run variance: $\sigma_\infty^2 = \omega / (1 - \alpha - \beta) = 0.000002 / (1 - 0.08 - 0.91) = 0.0002$

Long-run volatility: $\sigma_\infty = \sqrt{0.0002} \approx 1.41\%/\text{day}$ ✓

Persistence: $\alpha + \beta = 0.99$ (close to 1 = very slow mean-reversion)

On a day following a 3% return ($r_{t-1}^2 = 0.0009$):

$$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2 = 0.000002 + 0.08 \times 0.0009 + 0.91 \times 0.0002 = 0.000274$$

$\sigma_t = 1.66\%/\text{day}$ — 18% higher than the long-run average. Turbulence persists.

Formal Name — the mathematics

The **GARCH(1,1)** model (Bollerslev 1986) specifies:

$$r_t = \mu + \varepsilon_t, \quad \varepsilon_t = \sigma_t z_t, \quad z_t \stackrel{iid}{\sim} F(0,1)$$

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

Constraints: $\omega > 0$, $\alpha \geq 0$, $\beta \geq 0$, $\alpha + \beta < 1$ (covariance-stationarity). Persistence = $\alpha + \beta$ (close to 1 in BRICS markets: slow volatility mean-reversion).

Gaussian QMLE: we fit assuming $z_t \sim \mathcal{N}(0,1)$ even when z_t has heavy tails. This gives consistent estimates of (ω, α, β) regardless of the true distribution. The residuals $\hat{\varepsilon}_t/\hat{\sigma}_t$ will exhibit excess kurtosis if the true conditional distribution is non-Gaussian (stylized fact 7).

Where the Analogy Breaks

GARCH is a parametric model. It assumes a specific mathematical form for how today's variance relates to yesterday's. Real markets show leverage effects (negative returns increase volatility more than positive returns of the same size), jump components, and long-memory volatility that GARCH(1,1) cannot fully capture. GARCH is a diagnostic tool here, not a generative model.

Why It Matters Here — link to the thesis

GARCH appears in two places: (1) **Conditional heavy-tail metric** — we fit Gaussian QMLE, extract residuals $\varepsilon_t/\hat{\sigma}_t$, and measure residual excess kurtosis; a good generator's synthetic residuals should have similar kurtosis. (2) **Downstream utility (TSTR)** — we fit GARCH on synthetic data, use the estimated parameters to filter real data, and compute conditional VaR backtest results.

2.4 ARCH Test

Situation — the everyday picture

You want to know whether a return series has volatility clustering or not. The ARCH-LM test gives a statistical answer: it checks whether squared past returns predict current squared returns.

Worked Arithmetic — concrete numbers

ARCH-LM regression: regress r_t^2 on $r_{t-1}^2, \dots, r_{t-p}^2$ and test R^2 .

LM statistic = $n \cdot R^2 \sim \chi^2(p)$ under the null of no ARCH.

Simulated GARCH data ($n = 1,200$): LM ≈ 56 –125, p -value ≈ 0.0 .

Real BRICS data (real series): LM ≈ 51 , p -value = 0.0.

Real BRICS data (shuffled series): LM ≈ 67 , p -value = 0.0.

The test saturates: both real and shuffled series give $p = 0.0$. The p -value difference $|\Delta p| = 0.000000$. The ARCH-LM p -value cannot distinguish a good generator from a shuffled deck on real BRICS data.

Formal Name — the mathematics

Engle's (1982) ARCH-LM statistic for p lags:

$$LM = n \cdot R_{\text{aux}}^2, \quad \text{where aux: } r_t^2 = \gamma_0 + \sum_{j=1}^p \gamma_j r_{t-j}^2 + \eta_t$$

Under H_0 (no ARCH): $LM \xrightarrow{d} \chi^2(p)$.

Where the Analogy Breaks

The ARCH-LM test is powerful in theory and in simulation (99% correct on simulated GARCH). It fails on real fat-tailed data because the squared-return regression is dominated by extreme values. Both real and shuffled series have the same squared-return distribution, so both produce large LM statistics and zero p -values. The test is correctly described as “saturated on real data”.

Why It Matters Here — link to the thesis

Both `arch_pvalue_diff` and `arch_stat_diff` are placed in **DESCRIPTIVE_COLS** — computed and displayed for diagnostic transparency, but excluded from all composite ranking scores. ACF-MAE on $|r_t|$ is the primary volatility-clustering ranking signal because it never saturates.

2.5 Hurst Exponent and Long Memory

Situation — the everyday picture

The Nile River's floods are not independent year to year. A decade of high floods tends to be followed by another decade of high floods. This is *long memory*: correlations that decay so slowly they never fully disappear.

Worked Arithmetic — concrete numbers

For a process with long memory, the R/S statistic (range divided by standard deviation) scales as:

$$\mathbb{E}[R_n/S_n] \approx c \cdot n^H$$

For independent data: $H = 0.5$ exactly. For BRICS $|r_t|$ (volatility proxy): $H \approx 0.57\text{--}0.65$ typically.

Estimation at BOVESPA ($n = 4,953$): divide series into m blocks of size n/m , compute R/S for each, regress $\log(R/S)$ vs $\log(n)$, slope = $\hat{H}$.

Standard deviation: ≈ 0.022 at $n = 1,000$; ≈ 0.004 at $n = 4,000$. At 5-year data ($n = 995$): $\hat{H} \pm 0.022$ — barely significant above $H = 0.5$. At 20-year data ($n = 4,960$): $\hat{H} \pm 0.004$ — clearly significant.

Formal Name — the mathematics

The **Hurst exponent** $H \in (0, 1)$ characterises long-range dependence:

- $H = 0.5$: independent (random walk, Brownian motion)
- $H > 0.5$: positively correlated at long lags (persistent, long memory)

- $H < 0.5$: negatively correlated at long lags (anti-persistent, mean-reverting)

Applied to $|r_t|$, $H > 0.5$ implies that large-volatility periods have a tendency to persist longer than an independent process would predict (fact 4: long memory in volatility).

Where the Analogy Breaks

The R/S estimator is biased in small samples and sensitive to non-stationarities. The 20-year data substantially reduces this bias. We apply H to $|r_t|$, not r_t itself, because raw returns have $H \approx 0.5$ (the near-zero ACF fact) while volatility has $H > 0.5$.

Why It Matters Here — [link to the thesis](#)

$\text{hurst_diff} = |H_{\text{real}} - H_{\text{syn}}|$ is in the **Temporal** family. The shuffled control achieves Hurst diff ≈ 0.052 (correctly penalised: shuffling destroys long memory). A good generator should reproduce $H \approx 0.6$ for the absolute return series of each market.

Chapter 3

Heavy Tails and Power Laws

3.1 Power Laws

Situation — the everyday picture

City sizes: New York has 8 million people. The next three cities have 4M, 3M, 2M. Cut the population threshold in half, and twice as many cities qualify. This doubling relationship is a *power law*: the number of cities with population $> x$ scales like $x^{-\alpha}$ for some exponent α .

Problem — where it breaks or why it matters

The normal distribution has *exponential* tails: the probability of a 10σ event is not just much smaller than a 5σ event — it is impossibly smaller (ratio $\approx 10^{17}$). Power-law tails decay much more slowly, making extreme events genuinely probable on financial timescales.

Worked Arithmetic — concrete numbers

NIFTY50 daily returns (20-year, $n = 4,954$):

The top 5% of $|r_t|$ values: 248 observations, smallest = 1.97%, largest = 15.91%.

Hill estimate: $\hat{\alpha} = k / \sum_{i=1}^k \ln(|r|_{(n-i+1)} / |r|_{(n-k)})$

With $k = \lfloor 0.05n \rfloor = 248$: $\hat{\alpha} \approx 2.8$.

Implication: $\mathbb{P}(|r| > x) \approx C \cdot x^{-2.8}$.

At $x = 5\sigma$: normal predicts 5.7×10^{-7} ; power-law predicts $\approx (5)^{-2.8} \times C$.

If $C = 0.05 \times 5^{2.8} \approx 3.3$: $\mathbb{P} \approx 3.3 \times 5^{-2.8} \approx 0.022 = 2.2\%$ vs Gaussian's 0.00006%.

Formal Name — the mathematics

A random variable X has a **power-law tail** with index $\alpha > 0$ if:

$$\mathbb{P}(|X| > x) \sim L(x) \cdot x^{-\alpha} \quad \text{as } x \rightarrow \infty$$

where $L(x)$ is a slowly varying function (converges to a constant). The **tail index** α determines which moments exist:

$$\mathbb{E}[|X|^k] < \infty \iff k < \alpha$$

With BRICS $\hat{\alpha} = 2.54$ – 3.21 : variance exists ($k = 2 < \alpha$), skewness is borderline ($k = 3$: exists only for $\hat{\alpha} > 3$, i.e., only BOVESPA marginally), kurtosis does not exist anywhere ($k = 4 > \hat{\alpha}$ everywhere).

Where the Analogy Breaks

“Tail index” is an asymptotic property — it describes what happens in the extreme tail, not the bulk of the distribution. In finite samples (even 20-year daily data), the Hill estimator averages over both the power-law region and the Gaussian-like body of the distribution. The estimate is sensitive to the choice of k (number of order statistics used).

Why It Matters Here — [link to the thesis](#)

`tail_index_diff` = $|\hat{\alpha}_{\text{real}} - \hat{\alpha}_{\text{syn}}|$ is the primary heavy-tail metric in the **Fidelity** family. This replaces `kurtosis_diff`, which is undefined for $\hat{\alpha} < 4$. The Hill estimator has $\text{sd} \approx 0.035$ at $n = 4,960$ — precise enough to distinguish generators that reproduce the tail index from those that do not.

3.2 Why Kurtosis Is Unreliable for BRICS Markets

Situation — the everyday picture

You want to measure how “extreme” a dataset is. The obvious measure is kurtosis: the fourth standardised moment. A larger kurtosis means more extreme events. You measure kurtosis on 250 days of BOVESPA data. Then on 2,000 days. The number changes dramatically.

Worked Arithmetic — concrete numbers

BOVESPA kurtosis at different sample sizes (demonstrated in the session):

Sample n	Sample kurtosis	Hill $\hat{\alpha}$
250	2.53	4.83 (sd 1.253)
2000	11.84	3.11 (sd 0.316)

Sample kurtosis *increases* as n grows — diverging rather than converging. Hill $\hat{\alpha}$ *converges* as n grows.

Mathematical reason: if $\mathbb{E}[X^4] = \infty$ (which occurs when $\hat{\alpha} < 4$), the sample kurtosis $\hat{\kappa}_n = \frac{n^{-1} \sum x_i^4}{(n^{-1} \sum x_i^2)^2}$ diverges almost surely as $n \rightarrow \infty$.

Formal Name — the mathematics

For a distribution with tail index α :

$$\mathbb{E}[|X|^k] < \infty \iff k < \alpha$$

When $\alpha < 4$ (as in all BRICS markets): $\mathbb{E}[X^4] = \infty$, so the population kurtosis is undefined. The **Law of Large Numbers** requires finite expectation; without it, the sample mean of X^4 does not converge. The sample kurtosis $\hat{\kappa}_n$ grows without bound, following a stable distribution with index $\alpha/4 < 1$.

Contrast with Hill: the Hill estimator uses log spacings of order statistics, which are $\Gamma(1, 1)$ distributed under the power-law model. These have finite moments of all orders, making the Hill estimator consistent and asymptotically normal regardless of α .

Where the Analogy Breaks

“Sample kurtosis = 65.70 for MOEX” is not a measurement of the population kurtosis — the population kurtosis is infinite. It is an artifact of sample size, driven by the handful of extreme observations (especially 2022-02-24: −33.3%). If we took a different 20-year window, the sample kurtosis might be 30 or 150. The Hill estimate $\hat{\alpha} \approx 2.7$ for MOEX is comparatively stable.

Why It Matters Here — [link to the thesis](#)

This is the core motivation for TASK 17 in the pipeline. Kurtosis_diff and skewness_diff are demoted from FIDELITY_COLS (7 metrics, ranked) to DESCRIPTIVE_COLS (7 metrics, displayed but not ranked). tail_index_diff takes their role as the primary heavy-tail fidelity metric. The length guard in `FinancialMetrics.__init__` warns when comparing series of different lengths, because kurtosis differences between unequal-length series are biased by length alone.

3.3 Wasserstein Distance

Situation — the everyday picture

Imagine two piles of sand on a table. One pile represents the distribution of real returns; the other represents synthetic returns. You want to measure how different they are. The Wasserstein distance asks: what is the minimum amount of “work” (mass times distance) needed to move one pile into the shape of the other?

Problem — where it breaks or why it matters

The Kolmogorov-Smirnov (KS) test measures the maximum gap between two CDFs. It was designed for independent data. Market returns are not independent (ACF of $|r_t| > 0$ for 100+ lags). For correlated data, the effective sample size is $n_{\text{eff}} = n / (1 + 2 \sum_k \rho_k) \ll n$. KS uses the nominal n , making its p -values severely anti-conservative.

Worked Arithmetic — concrete numbers

For one-dimensional distributions, the Wasserstein-1 distance is the L_1 distance between quantile functions:

$$W_1(P, Q) = \int_0^1 |F_P^{-1}(u) - F_Q^{-1}(u)| \, du$$

Practical computation: sort both series in ascending order. Pair each real return with its rank-matched synthetic return. Average absolute difference.

For the shuffled control: $W_1 = 0.000e+00$ (exact zero). The shuffled series is a permutation of the real data; sorted, they are identical.

For a well-trained generator: $W_1 \approx 0.001\text{--}0.005$ (residual fitting error).

Formal Name — the mathematics

The **Wasserstein- p distance** between distributions P and Q :

$$W_p(P, Q) = \left(\inf_{\gamma} \mathbb{E}_{(X, Y) \sim \gamma} \|X - Y\|^p \right)^{1/p}$$

where the infimum is over joint distributions (couplings) with marginals P and Q . For $p = 1$ in 1D: reduces to the integral of the absolute difference of CDFs:

$$W_1(P, Q) = \int_{-\infty}^{\infty} |F_P(x) - F_Q(x)| dx$$

Where the Analogy Breaks

Wasserstein is permutation-invariant: $W_1(\text{real}, \text{shuffled}) = 0$ because sorting both produces the same sequence. This is a feature (it correctly measures distributional proximity) and a limitation (it does not measure temporal proximity). This is precisely why it sits in FIDELITY rather than TEMPORAL.

Why It Matters Here — [link to the thesis](#)

`wasserstein` is one of the 7 Fidelity metrics. The shuffled-control audit revealed that Wasserstein, like all Fidelity metrics, gives a perfect score to the shuffled control. This motivated the three-score composite design: Fidelity metrics measure “do the individual values look right?”; Temporal metrics measure “does the ordering look right?”.

Chapter 4

Generative Adversarial Networks

4.1 The GAN Idea

Situation — the everyday picture

A counterfeiter tries to make fake currency. A bank teller learns to detect fakes. Each tries to fool or catch the other. Over time, both get better — the counterfeiter produces more convincing fakes, the teller develops sharper detection skills. Eventually, the fakes may be indistinguishable from real currency.

Problem — where it breaks or why it matters

We want a machine that generates realistic market return sequences. A single neural network trained directly is hard to evaluate: how do you define a loss function that measures “realism” for a complex, high-dimensional sequence? The GAN answer: train a second network to play the role of the expert bank teller. It provides the loss signal automatically.

Formal Name — the mathematics

A **GAN** (Goodfellow et al., 2014) consists of two neural networks:

- **Generator** $G : \mathcal{Z} \rightarrow \mathcal{X}$: maps random noise $z \sim p_z$ to a synthetic sample $\tilde{x} = G(z)$
- **Discriminator** $D : \mathcal{X} \rightarrow [0, 1]$: classifies inputs as real ($D(x) = 1$) or synthetic ($D(\tilde{x}) = 0$)

Training is a minimax game:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

At the Nash equilibrium: $p_G = p_{\text{data}}$ and $D(x) = 1/2$ everywhere.

Where the Analogy Breaks

The Nash equilibrium is a theoretical ideal. In practice, GAN training is unstable: mode collapse (generator produces only a few outputs), discriminator dominance (generator gradients vanish), and oscillation without convergence are common. WGAN-GP addresses the most severe of these issues.

Why It Matters Here — link to the thesis

All three generators in this thesis (TimeGAN, QuantGAN, FinGAN) are GANs. The GAN framework is chosen because it implicitly defines the target distribution through adversarial training rather than requiring an explicit density. This is essential for market returns, whose true density is unknown and non-Gaussian.

4.2 WGAN and the Wasserstein Distance**Situation — the everyday picture**

The original GAN discriminator outputs a probability $\in [0, 1]$. When the generator's distribution is far from the real one, the discriminator saturates: it assigns near-zero probability to fake samples, and the gradient of $\log(1 - D(G(z)))$ vanishes. The generator cannot learn which direction to improve.

Problem — where it breaks or why it matters

Vanishing gradients in early training. Mode collapse (generator produces a single realistic example to fool the fixed discriminator). Training instability. These problems worsen when the real and generated distributions do not overlap — which is always the case early in training.

Formal Name — the mathematics

Wasserstein GAN (Arjovsky et al., 2017) replaces the discriminator with a **critic** $f : \mathcal{X} \rightarrow \mathbb{R}$ (no sigmoid) and minimises the Wasserstein-1 distance:

$$W_1(p_{\text{data}}, p_G) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p_{\text{data}}} [f(x)] - \mathbb{E}_{z \sim p_z} [f(G(z))]$$

by the Kantorovich-Rubinstein duality, where the supremum is over 1-Lipschitz functions.

WGAN-GP (Gulrajani et al., 2017) enforces the Lipschitz constraint via a gradient penalty:

$$\mathcal{L}_{GP} = \lambda \cdot \mathbb{E}_{\hat{x} \sim p_{\hat{x}}} \left[(\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1)^2 \right]$$

where $\hat{x} = \varepsilon x_{\text{real}} + (1 - \varepsilon)x_{\text{fake}}$, $\varepsilon \sim U[0, 1]$. Standard values: $\lambda = 10$, $n_{\text{critic}} = 5$.

Why $n_{\text{critic}} = 5$: the critic must accurately estimate W_1 before providing a useful gradient to the generator. Fewer updates underfit the critic, giving a biased gradient signal. 5 is the empirically validated minimum (Gulrajani et al., 2017).

Why $\beta_1 = 0$ in Adam: in adversarial training, the correct gradient direction for the generator reverses each time the critic is updated. Momentum ($\beta_1 > 0$) accumulates the previous direction and corrupts the current gradient. $\beta_1 = 0$ uses only the current gradient.

Where the Analogy Breaks

The gradient penalty is computed at interpolated points, not directly on training data. This means the Lipschitz constraint is only approximately enforced (not everywhere). In practice, WGAN-GP is far more stable than weight clipping (Arjovsky 2017) or spectral normalisation, and remains the standard.

Why It Matters Here — [link to the thesis](#)

Both QuantGAN and FinGAN use WGAN-GP with $n_{\text{critic}} = 5$, $\lambda_{gp} = 10$, Adam $\beta_1 = 0$, $\beta_2 = 0.9$. These hyperparameters are floors, not arbitrary choices — using $n_{\text{critic}} = 1$ or $\lambda_{gp} = 1$ would invalidate the Wasserstein approximation. This is documented in ED Table 3 of the paper.

4.3 TimeGAN

Situation — the everyday picture

You want to teach a machine to write convincing diary entries. First, you teach it to encode each entry into a compressed representation and decode back (autoencoder). Then you teach it to predict the next entry from the previous ones (supervisor). Only after these skills are established do you introduce the adversarial game.

Formal Name — the mathematics

TimeGAN (Yoon et al., 2019) has four components:

- **Embedder** $e : \mathcal{X} \rightarrow \mathcal{H}$: encodes real sequences to latent space
- **Recovery** $\hat{r} : \mathcal{H} \rightarrow \mathcal{X}$: decodes latent back to sequence
- **Supervisor** $s : \mathcal{H} \rightarrow \hat{\mathcal{H}}$: predicts next latent step from current
- **Generator** $G : \mathcal{Z} \rightarrow \mathcal{H}$: maps noise to latent sequences

Four training phases:

1. **Phase 1 — Autoencoder:** $\min \|X - \hat{r}(e(X))\|^2$
2. **Phase 2 — Supervisor:** $\min \|H_{t+1} - s(H_t)\|^2$ (temporal causality in latent space)
3. **Phase 3 — Joint adversarial:**

$$\mathcal{L}_G = \mathcal{L}_U + 100\sqrt{\mathcal{L}_S} + 100\mathcal{L}_V$$

where $\mathcal{L}_U$ = unsupervised GAN loss, $\mathcal{L}_S$ = supervised latent loss, $\mathcal{L}_V$ = moment-matching on mean/std

4. **Phase 4 — Fine-tuning:** refine recovery with generated sequences

Backbone: GRU (Gated Recurrent Unit). Hidden dim = 24, 3 layers, lr = 10^{-3} .

Where the Analogy Breaks

The latent-space supervisor is the key innovation: by training the supervisor to predict the next latent state, TimeGAN forces the latent space to encode temporal causality. However, this requires many gradient steps per phase to work. With `epochs=5` (as in the current smoke-test pipeline), each phase gets approximately 2 gradient steps — far too few for the GRU to form any useful representation. TimeGAN results in the current draft should be interpreted as lower bounds.

Why It Matters Here — [link to the thesis](#)

TimeGAN's poor temporal ranking in the current results reflects training budget, not architectural weakness. The recommended training is $\geq 5,000$ iterations per phase (or ≥ 50 epochs at the current batch size). The RunPod production run will use adequate epochs; the current paper's TimeGAN results are explicitly flagged as undertrained.

4.4 QuantGAN

Situation — the everyday picture

Instead of processing a sequence step-by-step (like a recurrent network), imagine looking at the whole sequence at once with multiple “zoom levels”: a short zoom sees the last 2 days, a medium zoom sees the last week, a long zoom sees the last month. The network combines all zoom levels simultaneously to capture structure at every timescale.

Formal Name — the mathematics

QuantGAN (Wiese et al., 2020) uses a **Temporal Convolutional Network** (TCN) with *causal dilated convolutions*:

- **Causal:** the convolution at time t only sees inputs at times $\leq t$ (no future leakage)
- **Dilated:** dilation d means the filter sees steps $t, t-d, t-2d, \dots$ (stride- d subsampling in the receptive field)

With K residual blocks of dilations $1, 2, 4, \dots, 2^{K-1}$, the receptive field = 2^K steps.

Three TCN blocks with dilations 1/2/4: receptive field = 8 steps. For a 128-step sequence, this covers the full window.

WGAN-GP with $n_{\text{critic}} = 5$, $\lambda_{gp} = 10$. Noise dim = 100, lr = 10^{-4} .

Where the Analogy Breaks

The parallel multi-scale processing is the key advantage over RNNs: a GRU must “remember” information across many steps using a hidden state of fixed size, leading to gradient issues for long horizons. A dilated TCN accesses all timescales in one forward pass without vanishing gradients. The tradeoff: TCN is less natural for autoregressive modelling (each step predicts the next), and its inductive bias may over-smooth long-range structure.

Why It Matters Here — link to the thesis

QuantGAN’s TCN architecture is well-suited to the multi-scale structure of BRICS volatility: intraday effects (1–5 days), weekly patterns (5 days), and monthly cycles (20 days) are all covered by dilations 1/2/4 in a 128-step window. QuantGAN achieves the best composite ranking in current (smoke-test) results.

4.5 FinGAN

Situation — the everyday picture

Image-generating GANs start from a small noisy image (4×4 pixels) and progressively upsample it to full resolution (256×256), adding detail at each step. FinGAN adapts this idea to sequences: start from a short noise vector and upsample via transposed convolutions to a full 128-step return sequence.

Formal Name — the mathematics

FinGAN (this paper): CNN deconvolution WGAN-GP.

- Generator: $3 \times \text{ConvTranspose1d}$ with stride 2, base channels = 64
- Critic: mirror architecture with **Conv1d** and LayerNorm (not BatchNorm)

- `seq_len` must be divisible by 8 ($= 2^3$ from 3 upsampling blocks)

Why LayerNorm in the critic, not BatchNorm: the WGAN-GP gradient penalty is computed at a single interpolated point $\hat{x}$. BatchNorm normalises across the batch, introducing a dependency on other batch members that corrupts the per-point gradient norm calculation. LayerNorm normalises across channels at each point independently.

Where the Analogy Breaks

The image-GAN inductive bias assumes hierarchical structure: coarse features first, fine details added later. Market return sequences may not have this structure — the fine-scale noise on any given day is not a refinement of the coarse-scale trend. This may explain FinGAN's weaker performance relative to QuantGAN, whose temporal convolutions are better suited to sequential dependencies.

Why It Matters Here — [link to the thesis](#)

FinGAN serves as a baseline for CNN-based approaches to financial time series. Its architecture is deliberately simple (3 upsampling blocks, no attention, no recurrence) to isolate the effect of the upsampling inductive bias. The `seq_len` divisibility-by-8 constraint must be checked before training on any new dataset.

Chapter 5

Evaluation: Metrics, Taxonomy, and the Shuffled-Control Audit

5.1 The Metric Taxonomy

Situation — the everyday picture

You built a machine that generates fake playing cards. You want to check whether the fakes are good. You could count the card values (does the fake deck have the right number of aces?) — that’s a *fidelity* check. But you could also shuffle the deck and see if the pattern looks natural — that’s a *temporal* check. Both are necessary; neither alone is sufficient.

Formal Name — the mathematics

We classify all evaluation metrics into three groups:

Fidelity metrics (7, permutation-invariant): measure the marginal distribution of r_t only. A metric m is permutation-invariant if $m(\pi(r)) = m(r)$ for any permutation π . These metrics score the shuffled control perfectly by construction.

- **mean_diff**: $|\bar{r}_{\text{real}} - \bar{r}_{\text{syn}}|$
- **std_diff**: $|\text{sd}(r_{\text{real}}) - \text{sd}(r_{\text{syn}})|$
- **wasserstein**: $W_1(p_{\text{real}}, p_{\text{syn}})$
- **quantile_mse**: $K^{-1} \sum_k (Q_{\text{real}}(\alpha_k) - Q_{\text{syn}}(\alpha_k))^2$
- **tail_index_diff**: $|\hat{\alpha}_{\text{real}} - \hat{\alpha}_{\text{syn}}|$ (Hill estimator, top 5%)
- **extreme_events_diff**: $|P_{\text{real}}(|r| > 2\sigma) - P_{\text{syn}}(|r| > 2\sigma)|$
- **energy_distance**: $2\mathbb{E}\|X - Y\| - \mathbb{E}\|X - X'\| - \mathbb{E}\|Y - Y'\|$

Temporal metrics (7, ordering-sensitive): measure properties that depend on the sequence of r_t . The shuffled control scores poorly on all of these.

- **acf_returns_mae**: ACF-MAE of r_t at lags 1–20 (tests fact 2)
- **acf_absolute_mae**: ACF-MAE of $|r_t|$ (tests volatility clustering, fact 3)
- **acf_squared_mae**: ACF-MAE of r_t^2 (tests ARCH effects, fact 6)
- **hurst_diff**: $|H_{\text{real}} - H_{\text{syn}}|$ applied to $|r_t|$ (tests fact 4)
- **resid_kurtosis_diff**: $|\kappa_{\text{real}} - \kappa_{\text{syn}}|$ for GARCH residuals (tests fact 7)
- **discriminative_auc_dist**: $|AUC - 0.5|$ from logistic classifier
- **discriminative_auc_absz**: $|(AUC - 0.506)/0.084|$ (z-score vs empirical null)

Descriptive metrics (7, excluded from composite): computed and displayed for diagnosis only.

- **kurtosis_diff**: $\mathbb{E}[X^4] = \infty$ for $\hat{\alpha} < 4$; sample values diverge with n

- `skewness_diff`: $\mathbb{E}[X^3] = \infty$ for $\hat{\alpha} < 3$ (4/5 BRICS markets)
- `arch_pvalue_diff`: saturates on real data ($|\Delta p| = 0.000000$)
- `arch_stat_diff`: null sd 46.4 on simulated GARCH; too noisy to rank
- `var_coverage_error`: unconditional VaR; cannot distinguish Gaussian noise from real
- `garch_persistence_diff`: sd 0.31 across seeds; range 0.000–0.985
- `discriminative_auc_raw`: raw AUC value (superseded by dist/absz variants)

Why It Matters Here — [link to the thesis](#)

The three-group taxonomy is the central methodological contribution of this paper. The composite score = (fidelity_rank + temporal_rank)/2 gives equal weight to both families regardless of how many individual metrics each contains. The shuffled control is always shown in the results table with NaN ranks — it serves as the adversarial baseline.

5.2 The Shuffled-Control Audit

Situation — the everyday picture

You buy a set of playing cards supposedly shuffled randomly. To test the shuffler, you check: (1) are the right cards present (4 of each value)? (2) is the ordering random? Test (1) passes trivially — you can count cards without looking at order. Test (2) is the harder test that actually verifies the shuffle.

Worked Arithmetic — concrete numbers

Shuffled control construction:

```
rng = numpy.random.default_rng(42)
```

```
shuffled = rng.permutation(pooled_real_array).copy()
```

Scores of shuffled control on	
Metric	Shuffled control
wasserstein	0.000e+00 (exact zero)
kurtosis_diff	1.78×10^{-15} (floating-point noise)
energy_distance	2.97×10^{-5}
mean_diff	$< 10^{-14}$
quantile_mse	$< 10^{-14}$
acf_absolute_mae	0.076 (correctly penalised)
hurst_diff	0.052 (correctly penalised)

Under the old unweighted 19-metric composite: avg_rank = 1.24 (shuffled) vs 2.47 (real-like generator). The shuffled deck won.

Formal Name — the mathematics

A metric m is **permutation-invariant** if for all $r \in \mathbb{R}^n$ and all permutations $\pi \in S_n$:

$$m(\pi(r_1, \dots, r_n)) = m(r_1, \dots, r_n)$$

Any metric that depends only on the *sorted* values (empirical CDF, moments, quantiles, order statistics) is permutation-invariant. This includes: Wasserstein, energy distance, KS statistic, kurtosis, skewness, mean, variance, tail index, extreme event rate.

A metric m is **ordering-sensitive** if $\exists r, \pi$ such that $m(\pi(r)) \neq m(r)$. ACF, Hurst exponent, GARCH-based metrics, and the discriminative classifier (which uses time-window features) are all ordering-sensitive.

Where the Analogy Breaks

The shuffled control is not a bad generator — it is the best possible fidelity generator (it *is* the real data). Its failure on temporal metrics is not a bug; it is the point. Any evaluation that does not include the shuffled control as a baseline cannot claim to have tested temporal structure.

Why It Matters Here — [link to the thesis](#)

The shuffled control is a permanent entry in the evaluation table (with NaN composite rank). It plays the same role as a positive control in a biology experiment: it is expected to score well on fidelity and poorly on temporal. A GAN that scores worse than the shuffled control on temporal metrics has learned nothing about market dynamics.

5.3 Discriminative AUC: Null Calibration and Direction Bug**Situation — the everyday picture**

You train a classifier to tell real returns from synthetic returns. If it achieves 50% accuracy (random guessing), the synthetic data is indistinguishable from real. If it achieves 90%, the synthetic data is clearly different.

Worked Arithmetic — concrete numbers

Direction bug: under ascending rank (lower is better), $AUC = 0.30$ ranked above $AUC = 0.50$. But $AUC = 0.50$ is *better* (indistinguishable), and $AUC = 0.30$ is worse (the classifier is *anti-predictive* — it can tell real from fake by doing the opposite of its prediction). Fix: rank on $|AUC - 0.5|$.

Null calibration: even with two samples from the same distribution, the classifier achieves $AUC \neq 0.5$ exactly. Empirical null (15 real-vs-real half-splits): 0.506 ± 0.084 .

An observed $AUC = 0.62$: $z = (0.62 - 0.506)/0.084 = 1.36$, $p = 0.17$. Not significant.

CV shuffle bug: 20-day windows overlap by 19 observations. Adding `shuffle=True` to the K-fold cross-validation puts near-duplicate windows in both train and test, creating look-ahead leakage. Measured null shift: $0.506 \rightarrow 0.584$ with shuffled CV. Unshuffled CV is correct.

Formal Name — the mathematics

AUC (Area Under the ROC Curve) for a binary classifier: $\mathbb{P}(\hat{p}(x_{\text{real}}) > \hat{p}(x_{\text{syn}}))$.

The optimal value for a generator is $AUC = 0.5$ (chance level).

Our two ranking metrics:

$$\text{discriminative_auc_dist} = |AUC - 0.5|$$

$$\text{discriminative_auc_absz} = |(AUC - 0.506)/0.084|$$

Both are minimised at the GAN optimum and entered into the Temporal family.

Why It Matters Here — [link to the thesis](#)

The three bugs found in the discriminative AUC implementation (direction, null calibration, CV shuffling) are documented in ED Table 2 of the paper. They represent common pitfalls in

synthetic data evaluation that affect published benchmarks. The corrected implementation is in cell `aaef7bd2` of `3_4_integrated_pipeline.ipynb`.

Chapter 6

Downstream Utility: VaR Backtesting and QLIKE

6.1 Value at Risk

Situation — the everyday picture

A bank risk manager wants to answer: “What is the most I should expect to lose on any single trading day, with 95% confidence?” The answer is the **Value at Risk** (VaR) at the 5% level. If the daily VaR is 2%, it means the bank should expect to lose more than 2% on at most 1 day in 20.

Worked Arithmetic — concrete numbers

Unconditional VaR (5th percentile of the empirical distribution): For BOVESPA 20-year data, the 5th percentile of $r_t \approx -2.7\%$. $\text{VaR}_{0.05} = -2.7\%$ means: on 5% of days, expect worse than a -2.7% return.

Conditional VaR (GARCH-based):

$$\begin{aligned}\hat{\sigma}_t^2 &= \hat{\omega} + \hat{\alpha}r_{t-1}^2 + \hat{\beta}\hat{\sigma}_{t-1}^2 \\ \text{VaR}_t &= \hat{\mu} + \hat{\sigma}_t \cdot z_{0.05}, \quad z_{0.05} = -1.645\end{aligned}$$

This VaR changes each day based on yesterday’s volatility. On a quiet day, $\text{VaR}_t \approx -1.5\%$. After a large move, $\text{VaR}_t \approx -3.5\%$.

Why unconditional VaR is useless as an evaluation metric: Gaussian iid noise (white noise with σ = historical vol) achieves coverage error = 0.0276. Real GARCH parameters on real data also achieve coverage error = 0.0259. The two are nearly identical because unconditional VaR tests only the marginal distribution, which is already covered by Wasserstein and KS.

Formal Name — the mathematics

Value at Risk at level α :

$$\text{VaR}_\alpha = \inf\{x : \mathbb{P}(r_t \leq x) \geq \alpha\}$$

For a GARCH model with normal innovations:

$$\text{VaR}_{\alpha,t} = \hat{\mu} + \hat{\sigma}_t \Phi^{-1}(\alpha)$$

Coverage error: $|\hat{v} - \alpha|$ where $\hat{v}$ = fraction of test days with $r_t < \text{VaR}_{\alpha,t}$. Correct model: $\hat{v} \approx \alpha = 5\%$. Observed for real GARCH on real BRICS data: $\hat{v} = 7.59\%$, so coverage error $= |0.0759 - 0.05| = 0.0259$.

Where the Analogy Breaks

VaR captures the 5th percentile only. It says nothing about what happens *beyond* the 5th percentile (Expected Shortfall / CVaR covers this). We use VaR because Kupiec and Christoffersen backtests are specifically designed for it, and because the thesis focuses on evaluating the GARCH structure, not the full tail.

Why It Matters Here — [link to the thesis](#)

The real-GARCH coverage error of 0.0259 is the reference baseline in `var_coverage_error`. A synthetic GARCH that transfers well should produce VaR forecasts with similar coverage. This metric is in DESCRIPTIVE (not TEMPORAL) because it probes only the marginal coverage rate, not the temporal dynamics of whether violations cluster together (which is what Christoffersen tests).

6.2 Kupiec Backtest

Situation — the everyday picture

If your VaR model is correct, exceedances (days where the loss exceeds VaR) should happen exactly 5% of the time. If you observe 40 exceedances in 200 test days (20%), something is wrong. The Kupiec test asks: is the observed exceedance rate statistically distinguishable from the claimed 5%?

Worked Arithmetic — concrete numbers

Let n = test days, x = observed exceedances, α = claimed coverage (5%).
Under the null hypothesis ($p = \alpha$):

$$LR_{\text{Kupiec}} = -2 \ln \left[\frac{\alpha^x (1 - \alpha)^{n-x}}{\hat{p}^x (1 - \hat{p})^{n-x}} \right], \quad \hat{p} = x/n$$

$LR_{\text{Kupiec}} \sim \chi^2(1)$ under H_0 .

Power at different n : $n = 125$ (5-year per-market): power = 17% at true $p = 7\%$ — barely above random. $n = 496$ (20-year per-market): power $\approx 56\%$. $n = 2,480$ (20-year pooled): power $\approx 99\%$.

With 5-year data, Kupiec is essentially powerless. With 20-year pooled data, it is meaningful.

Formal Name — the mathematics

Kupiec (1995) unconditional coverage test:

$$H_0 : p = \alpha, \quad H_1 : p \neq \alpha$$

Likelihood ratio statistic:

$$LR_{uc} = 2 \ln[\hat{p}^x (1 - \hat{p})^{n-x}] - 2 \ln[\alpha^x (1 - \alpha)^{n-x}] \sim \chi^2(1)$$

Rejects at 5% significance if $LR_{uc} > 3.84$.

Where the Analogy Breaks

Kupiec tests only the *unconditional* coverage: is the total fraction of exceedances right? It does not test whether exceedances cluster in time (which would indicate the model fails in precisely the turbulent periods that matter most). That is the Christoffersen test.

Why It Matters Here — link to the thesis

Kupiec p -values are reported in the downstream utility section but are descriptive only at per-market n . At pooled $n \approx 2,480$, they are informative. The primary ranking metric is QLIKE because it is a proper scoring rule with no minimum-sample-size issue.

6.3 Christoffersen Independence Test

Situation — the everyday picture

Your VaR model has 5% coverage (right number of exceedances). But all 5% happen in a single turbulent week. After that week, perfect calm. The model is correct on average but wrong when it matters: it fails during exactly the periods banks need accurate risk estimates.

Formal Name — the mathematics

Christoffersen (1998) independence test: tests whether VaR exceedances are independent over time.

Let $I_t = \mathbf{1}[r_t < \text{VaR}_{\alpha,t}]$ be the indicator of an exceedance. Define transition counts: n_{ij} = number of days where $I_{t-1} = i$ and $I_t = j$.

Under independence:

$$LR_{\text{ind}} = 2 \ln \left[\frac{\hat{\pi}_{01}^{n_{01}} (1 - \hat{\pi}_{01})^{n_{00}} \hat{\pi}_{11}^{n_{11}} (1 - \hat{\pi}_{11})^{n_{10}}}{\hat{\pi}^{n_{01} + n_{11}} (1 - \hat{\pi})^{n_{00} + n_{10}}} \right] \sim \chi^2(1)$$

where $\hat{\pi}_{ij} = n_{ij} / (n_{i0} + n_{i1})$ and $\hat{\pi} = (n_{01} + n_{11}) / (n_{00} + n_{01} + n_{10} + n_{11})$.

Why It Matters Here — link to the thesis

The combined Christoffersen test ($LR_{\text{cc}} = LR_{\text{uc}} + LR_{\text{ind}} \sim \chi^2(2)$) tests both unconditional coverage and independence simultaneously. For synthetic GARCH parameters to be useful, the VaR forecasts they produce must have both correct coverage and independent exceedances. This is a substantially harder bar than Kupiec alone.

6.4 QLIKE Loss

Situation — the everyday picture

You have two weather forecasters. Forecaster A says “tomorrow will be a 2% return day” (one number). Forecaster B says “tomorrow’s volatility will be 1.5%” (a distribution). You observe the actual return. How do you score Forecaster B, who gave you a distribution, not a point forecast?

Worked Arithmetic — concrete numbers

QLIKE (Patton 2011):

$$\text{QLIKE}_t = \log \hat{\sigma}_t^2 + r_t^2 / \hat{\sigma}_t^2$$

A volatility forecast $\hat{\sigma}_t$ that is too high penalises $\log \hat{\sigma}_t^2$ (unnecessarily large). A forecast too low penalises $r_t^2 / \hat{\sigma}_t^2$ (missed a large move).

TRTR (train and test on real data) baseline: QLIKE ≈ -8.134 , sd 0.000. A synthetic GARCH that produces parameters close to the true ones will have QLIKE close to -8.134 .

Per-market inversion (5-year data, $n \approx 126$): Gaussian noise: QLIKE = -6.888 . Real data: QLIKE = -6.876 . Gaussian noise beats real data — QLIKE is meaningless at this n . Shuffled control sd across seeds: 3.354 — random variation dominates.

Pooled (20-year, $n \approx 2,480$): QLIKE is stable and interpretable. Per-market ($n \approx 496$) also stable.

Formal Name — the mathematics

QLIKE (Quasi-Likelihood): a proper scoring rule for volatility forecasts.

$$\text{QLIKE}(r, \hat{\sigma}) = \frac{1}{T} \sum_{t=1}^T \left[\log \hat{\sigma}_t^2 + \frac{r_t^2}{\hat{\sigma}_t^2} \right]$$

A scoring rule is **proper** if it is minimised in expectation when $\hat{\sigma}_t^2 = \mathbb{E}[r_t^2 | \mathcal{F}_{t-1}]$ (the true conditional variance). Patton (2011) shows QLIKE is robust to the use of imperfect variance proxies (here: r_t^2 instead of the true σ_t^2).

Where the Analogy Breaks

QLIKE is minimised (more negative = better) when the volatility forecast matches the conditional variance. It cannot test whether the GARCH model is the right *specification* (e.g., whether leverage effects matter), only whether the estimated parameters are close to the true conditional variance. A synthetic GARCH with wrong parameters but right unconditional volatility might score well on QLIKE by accident — Kupiec and Christoffersen provide additional checks.

Why It Matters Here — link to the thesis

QLIKE is the primary downstream utility ranking signal because: (1) it is a proper scoring rule, (2) it directly tests whether the GARCH structure learned from synthetic data transfers to real data, and (3) it is continuous with no minimum-power issues at pooled $n \approx 2,480$. The TSTR protocol (Train on Synthetic, Test on Real) uses synthetic GARCH parameters to forecast real-data volatility and computes QLIKE on the real test set.

6.5 Walk-Forward Validation

Situation — the everyday picture

You want to evaluate a forecasting model on historical data. The naive approach: train on all data up to 2020, test on 2020–2026. But this uses only one test period. If 2022–2026 happened to be unusual (it was, for MOEX), your evaluation might be misleading.

Walk-forward validation uses multiple test periods, each with a fresh training set.

Formal Name — the mathematics**Rolling-origin walk-forward** (CTBench protocol):

- Divide data into K folds (here: $K = 5$)
- For fold k : train on data up to fold k 's start, test on fold k
- Report metric mean $\pm$ std across 5 folds

Unlike K -fold cross-validation, the order of folds is never shuffled. Later folds always use more training data (expanding window) or a shifted training window (rolling window). We use rolling origin.

Why not K -fold with shuffling? Financial returns have serial dependence. Shuffling folds puts future data in the training set (look-ahead leakage). The discriminative AUC case showed this explicitly: shuffling 20-day windows shifts the null from 0.506 to 0.584.

Why It Matters Here — [link to the thesis](#)

Five rolling folds per market give mean $\pm$ std across folds rather than a single metric estimate. This enables: (1) confidence intervals, (2) detection of fold-specific anomalies (MOEX fold 3 covers the 2022 invasion), (3) comparison of temporal stability across architectures. The CTBench protocol (Liao et al., 2025) requires this design for standardised reporting.

Symbol Glossary

Symbol	Meaning
r_t	Log return on day t : $\ln(P_t/P_{t-1})$
σ_t	Time-varying conditional standard deviation at t
σ	Unconditional (long-run) daily volatility
μ	Long-run mean return (close to zero)
α (GARCH)	ARCH coefficient: weight on the squared previous shock
β (GARCH)	GARCH coefficient: weight on previous variance
ω	GARCH intercept: ensures long-run variance is positive
$\hat{\alpha}$ (Hill)	Tail index estimate from Hill estimator
H	Hurst exponent ($H > 0.5$ implies long memory)
W_1	Wasserstein-1 (Earth Mover's) distance
$\mathbb{E}$	Expectation operator
Var	Variance operator
$\mathcal{N}(\mu, \sigma^2)$	Normal distribution with mean μ and variance σ^2
$\mathbb{P}(\cdot)$	Probability of an event
z_α	α -quantile of standard normal: $z_{0.05} = -1.645$
AUC	Area under the ROC curve
GAN	Generative Adversarial Network
WGAN-GP	Wasserstein GAN with Gradient Penalty
GRU	Gated Recurrent Unit
TCN	Temporal Convolutional Network
CNN	Convolutional Neural Network
BCE	Binary Cross-Entropy loss
GARCH	Generalised ARCH
VaR	Value at Risk
QLIKE	Quasi-likelihood loss for volatility forecast evaluation

Dependency graph. Every chapter in this knowledge base is self-contained, but builds on prior chapters as follows:

